



Mastering UI Design with Jetpack Compose – Days 15-16 Android 14 Masterclass



Liza Cheremisina / 6. June 2024 / Android



Mastering UI Design with Jetpack Compose – Days 15-16 Android 14 Masterclass

Welcome to Days 15-16 of our Android 14 Masterclass, focusing on Mastering UI Design with Jetpack Compose. These sessions have covered **essential UI components** like Scaffold layouts, TopAppBar, Sticky Headers, and more, along with advanced features like LazyRow and Icon Tinting. Our goal is to equip you with a solid understanding of Jetpack Compose for superior Android UI design.

1. Scaffold and TopAppBar in Jetpack Compose

The **Scaffold** layout provides a basic material design structure for an app's UI. It allows developers to easily implement app bars, drawers, floating action buttons, and more. A crucial part of this is the **TopAppBar**, which acts as the topmost bar in the app, often containing navigation icons, title, and additional actions.

TopAppBar Detailed Explanation:

- The **TopAppBar** typically contains a title and action icons.
- These icons are usually defined in the app's resources as drawables.
- Icons like **R.drawable.ic_account** and **ic_subscribe** are referenced to display specific images (e.g., an account icon or a subscription icon).

Implementing Icons in TopAppBar:

1. Icon Preparation:

- First, you need to have your icons ready in the **drawable** directory of your Android project. These icons are usually vector graphics, and their filenames would be something like **ic_account.xml**, **ic_subscribe.xml**, etc.

2. Referencing Icons:

- To use these icons in your **TopAppBar**, you reference them using their resource IDs like **R.drawable.ic_account**.

3. Using Strikethrough Naming:

- The notation **~~ic~~baseline_person_add_alt_1_24** suggests a strikethrough, possibly indicating that this specific naming convention or icon is no longer in use or has been renamed.

Example with Code

Here's a basic example of a `TopAppBar` with an action icon:

```
import androidx.compose.material.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.res.vectorResource

@Composable
fun MyAppBar() {
    TopAppBar(
        title = { Text("My Application") },
        navigationIcon = {
            IconButton(onClick = { /* Handle navigation icon click */ }) {
                Icon(ImageVector.vectorResource(R.drawable.ic_account),
                    contentDescription = "Account")
            }
        },
        actions = {
            // Action icon for subscription
            IconButton(onClick = { /* Handle subscribe icon click */ }) {
                Icon(ImageVector.vectorResource(R.drawable.ic_subscribe),
                    contentDescription = "Subscribe")
            }
            // Other actions can be added here
        }
    )
}
```

In this example:

- `R.drawable.ic_account` and `R.drawable.ic_subscribe` are used as the navigation icon and an action icon in the `TopAppBar`.

- The `IconButton` Composable is used for making the icons clickable.
- `ImageVector.vectorResource` is used to load vector drawable resources.

This is a simplified example. In a real-world application, you would handle icon clicks and possibly dynamically change icons based on the app's state. For beginners, understanding the basic structure and how to reference resources in Jetpack Compose is crucial.

2. Vector Assets and Asset Studio

Vector Assets

- **Vector Graphics in Android:** Vector assets are graphics defined in XML format, which allows them to scale without losing quality. This is particularly useful for supporting different screen sizes and densities in Android apps.
- **Advantages:** They are smaller in file size compared to bitmap images, and since they are scalable, you can use a single asset for multiple screen densities.
- **Usage:** These assets are commonly used for icons and simple graphics in Android apps.

Asset Studio

- **What is Asset Studio?** Asset Studio is a tool integrated into Android Studio that helps you create and manage vector assets (among other types of assets).
- **Functionality:** It allows you to import existing images (like SVGs or PNGs), modify them if necessary, and then convert them into the appropriate format for Android (typically XML for vectors).
- **Accessibility:** Asset Studio can be accessed from within Android Studio, making it a convenient tool for developers.

Using Vector Assets and Asset Studio

1. Creating a New Vector Asset:

- You can create a new vector asset by right-clicking on the `res` folder in Android Studio, then selecting **New → Vector Asset**. This opens the Vector Asset Studio wizard.

2. Importing and Editing:

- Within the Vector Asset Studio, you can choose to import a local SVG or PNG file or select from a collection of material icons provided by Android Studio.
- You have options to customize the imported asset, like changing its size, color, or opacity.

3. Generating the XML File:

- Once you finalize your asset, Asset Studio generates an XML file representing the vector graphic. This file is typically stored in the `res/drawable` directory.
- The XML file defines the vector graphic using vector path data, which Android can use to render the graphic at any size.

Example: Importing and Using a Vector Asset

1. Importing:

- In Android Studio, go to **File → New → Vector Asset**.
- Choose your source image or pick a material icon.
- Customize as needed and click **Finish**. This saves the asset in your project's `drawable` folder.

2. Using in Layouts or Compose:

- Once the vector asset is part of your project, you can reference it in your XML layouts or Jetpack Compose UIs.
- For example, in an XML layout, you might use it as `android:src="@drawable/ic_my_vector_asset"`.

- In Jetpack Compose, you would use `painterResource(id = R.drawable.ic_my_vector_asset)`.

3. Sticky Header in Jetpack Compose

- **What is a Sticky Header?** In UI design, a sticky header is a section that sticks to the top of the screen as the user scrolls through a list of items. When a new header comes into the view, it pushes the previous one out of the way.
- **Usage in Lists:** This is often used in long lists or scrolling content to keep the context or category label visible while browsing through different sections.

Implementation in Jetpack Compose:

- In Jetpack Compose, this effect is typically achieved in a `LazyColumn` or `LazyRow`, where `stickyHeader` is used to make a particular item act as a sticky header.

Example: Sticky Header with Text in LazyColumn

Here's a simple example demonstrating how to create a sticky header with text in Jetpack Compose:

```
@Composable
fun StickyHeaderTextExample() {
    LazyColumn {
        // Sticky header
        stickyHeader {
            Text(
                text = "Sticky Header",
                fontSize = 20.sp,
                // Additional styling
            )
        }
    }
}
```

```
// List items
itemsIndexed(
    listOf("Item 1", "Item 2", "Item 3") // Replace with your data
) { index, item ->
    Text(
        text = item,
        fontSize = 16.sp
        // Additional styling
    )
}
}
```

In this example:

- **stickyHeader** is used within **LazyColumn** to create a sticky header.
- **Text** composable inside **stickyHeader** displays the header text.
- The list items follow after the sticky header section.

In short, “Sticky Header” is a UI element within a scrolling list, where a specific text element remains fixed at the top of the screen until it’s pushed off by another element. This is a common feature in lists or scrolling content where you want to maintain a header or label visible until the next section comes into view.

4. LazyRow in Jetpack Compose

- **What is LazyRow?** **LazyRow** is a composable in Jetpack Compose that lays out its children in a horizontal sequence, similar to how **RecyclerView** works in traditional Android development. However, unlike **RecyclerView**, **LazyRow** is more straightforward to use in the declarative UI pattern of Compose.

- **Laziness:** The term “lazy” in `LazyRow` refers to its behavior of only composing and laying out the visible items on the screen. It efficiently recycles the off-screen elements, thereby optimizing performance, especially for long lists of items.

Key Features of LazyRow

- **Horizontal Scrolling:** As the name suggests, `LazyRow` arranges its children in a row, allowing for horizontal scrolling.
- **Efficient for Large Datasets:** It only renders the items that fit on the screen and recycles them as you scroll. This makes it efficient for handling large datasets.
- **Customizable Layouts:** You can customize the arrangement of items, spacings, and alignments.

Example Usage of LazyRow

Here’s a basic example to illustrate the usage of `LazyRow`:

```
@Composable
fun MyLazyRow() {
    LazyRow {
        items(listOf("Item 1", "Item 2", "Item 3")) { item ->
            // Replace Text with any composable to represent your item
            Text(item)
        }
    }
}
```

In this example:

- `LazyRow` is used to create a horizontally scrollable row.
- `items` is a function that takes a list and a lambda. The lambda defines how each item in the list should be displayed.

- **Text** is used here for simplicity, but in a real application, this could be any composable representing more complex UI elements.

Key Notes:

- **Understanding Composables:** In Jetpack Compose, everything is a composable function. Beginners should understand how composables work and how they can be combined to create complex UIs.
- **Flexibility:** **LazyRow** can be customized with various parameters to control the layout behavior, like spacing between items, padding, and alignment.
- **Performance Considerations:** Remember that **LazyRow** is designed for performance in lists with many items. Its lazy nature ensures that only the items needed for the current view are rendered, reducing memory usage and improving scroll performance.

LazyRow is a powerful tool in the Jetpack Compose toolkit, enabling developers to create horizontal list layouts easily and efficiently.

5. Icon Tint Property in Jetpack Compose

- **What is Tinting?** Tinting in the context of UI design refers to applying a color filter to an icon or graphic. In Jetpack Compose, the **tint** property is used to change the color of an **Icon** composable.
- **Purpose:** Tinting is often used to ensure icons match the app's color theme, to signify a selected or active state, or to provide visual feedback in response to user interactions.

Icon Composable in Jetpack Compose

- **Usage:** The **Icon** composable is a part of the Jetpack Compose UI toolkit used to display icons from different sources, such as resource files or image vectors.
- **Tint Property:** The **tint** property in the **Icon** composable is used to apply a color tint to the icon. It accepts a **Color** value.

Syntax and Examples

1. Basic Icon with Tint:

- Syntax:

```
Icon(  
    painter = painterResource(id = R.drawable.ic_your_icon),  
    contentDescription = "description",  
    tint = Color.Red  
)
```

This code displays an icon with the resource ID `R.drawable.ic_your_icon` and applies a red tint to it.

1. Dynamic Tint Based on State:

- Syntax:

```
val isSelected = remember { mutableStateOf(false) }  
  
Icon(  
    painter = painterResource(id = R.drawable.ic_your_icon),  
    contentDescription = "description",  
    tint = if (isSelected.value) Color.Green else Color.Gray  
)
```

This code changes the tint of the icon based on a state (`isSelected`). If the item is selected, the tint is green; otherwise, it's gray.

1. No Tint:

- **Syntax:**

```
Icon(  
    painter = painterResource(id = R.drawable.ic_your_icon),  
    contentDescription = "description",  
    tint = Color.Unspecified  
)
```

Using `Color.Unspecified` will apply no tint to the icon, displaying it in its original colors.

Key Notes:

- **Understanding Composables:** In Jetpack Compose, UI elements are created using composable functions. The `Icon` composable is used to display icons.
- **Modifying Appearance:** The `tint` property is a straightforward way to alter the appearance of icons to match your UI design.
- **Color Class:** The `Color` class in Compose is used to define color values. You can use predefined colors (like `Color.Red`) or define custom colors using ARGB values.

Understanding how to use the `tint` property effectively allows for more dynamic and responsive UI designs, making the app visually coherent and intuitive for users.

6. ModalBottomSheetLayout in Jetpack Compose

- **What is ModalBottomSheetLayout?** `ModalBottomSheetLayout` is a composable in Jetpack Compose used to implement a modal bottom sheet. This is a UI component that slides up from the bottom of the screen, covering the main content partially or fully. It's "modal" in the sense that it typically requires an action to be dismissed and can block interaction with the underlying content.

- **Use Cases:** Commonly used for menus, selectors, or to display additional information without navigating away from the current screen.

Key Features

- **Interactivity:** The bottom sheet can be dismissed by swiping down or by pressing the back button.
- **Customization:** You can customize its size, content, and how it interacts with the rest of the UI.
- **State Management:** The visibility and state of the bottom sheet are typically managed using state holders in Compose, like `ModalBottomSheetState`.

Implementation in Jetpack Compose

1. State Initialization:

- First, initialize the `ModalBottomSheetState`. This state controls the visibility and state (expanded or hidden) of the bottom sheet.

2. ModalBottomSheetLayout Composable:

- `ModalBottomSheetLayout` is wrapped around your main content. It takes two primary parameters: `sheetContent` (to define what appears in the bottom sheet) and `content` (for the main content of the screen).

Syntax and Examples

1. Basic Example:

```
@Composable
fun MyScreen() {
    val sheetState = rememberModalBottomSheetState(initialValue =
        ModalBottomSheetValue.Hidden)
```

```
ModalBottomSheetLayout(  
    sheetState = sheetState,  
    sheetContent = { Text("Here's some content in the bottom sheet") }  
) {  
    // Main content of your screen  
    Button(onClick = { sheetState.show() }) {  
        Text("Show Bottom Sheet")  
    }  
}
```

This example shows a basic implementation where pressing a button reveals the bottom sheet.

1. Handling State:

- You can control the bottom sheet's visibility by modifying **sheetState** (using **show()** or **hide()** methods).

2. Customizing Sheet Content:

- The **sheetContent** lambda allows you to define the content of the bottom sheet. This can be any composable content, from text and images to custom layouts.

Key Notes:

- **State and Composition:** Understanding how state works in Jetpack Compose is crucial, as it allows you to create responsive and interactive UIs.
- **Composable Functions:** Each UI element in Jetpack Compose, including **ModalBottomSheetLayout**, is a composable function, offering a flexible and declarative approach to UI design.
- **User Experience:** Consider the user experience when using modal bottom sheets. They should be used in a way that feels intuitive and not disruptive to the user flow.

ModalBottomSheetLayout is a versatile component in the Compose toolkit, enabling developers to add an additional layer of interaction to their app's UI in an efficient and user-friendly manner.

Conclusion: Mastering UI Design with Jetpack Compose – Days 15-16 Android 14 Masterclass

In these two days, we've explored the diverse capabilities of Jetpack Compose, from basic UI structures to interactive elements like ModalBottomSheetLayout. This journey through Jetpack Compose's features is crucial for designing sophisticated Android applications. As we wrap up, we hope these insights have enhanced your skills in creating dynamic and user-friendly Android UIs.

If you want to skyrocket your Android career, check out our [The Complete Android 14 & Kotlin Development Masterclass](#). Learn Android 14 App Development From Beginner to Advanced Developer.

Master Jetpack Compose to harness the cutting-edge of Android development, **gain proficiency in XML** — an essential skill for numerous development roles, and **acquire practical expertise by constructing real-world applications**.

Check out Day 13 of this course [here](#).

Check out Day 14 of this course [here](#).

Related Posts:

1. [Storing Data Permanently Part 1/2 – Day 13 Android 14 Masterclass](#)
2. [The Power of Jetpack Compose and UI Customization – Day 5 Android 14 Masterclass](#)
3. [Mastering State Management and Essential Kotlin Syntax – Day 6 Android 14 Masterclass](#)
4. [Storing Data Permanently Part 2/2 – Day 14 Android 14 Masterclass](#)

Tags: [ANDROID 14](#) [ASSET STUDIO](#) [COMPOSABLES](#) [DRAWABLE](#) [ICON TINT](#) [ISSELECTED](#)

[JETPACK COMPOSE](#) [KOTLIN](#) [KOTLIN SYNTAX](#) [LAZYCOLUMN](#) [LAZYROW](#)

[MODALBOTTOMSHEETSTATE](#) [SCAFFOLD](#) [STICKY HEADER](#) [TOPAPPBAR](#) [UI](#) [UI DESIGN](#)

[VECTOR ASSETS](#) [XML](#)

The ultimate boost for your coding skills

Get exclusive coding industry insights, special offers, and free resources directly into your inbox

[Sign me up](#)

Join Over **100,000+** incredible coders.

[Collaboration](#) [Media Kit](#) [Contact](#) [Imprint](#) [About Us](#) [Privacy Policy](#)

©2024 TutorialsEU

Neve | Powered by WordPress

